

API Authentication Vulnerabilities — Interview Prep Notes

1. Logical Description

- Auth = proving identity (vs authz = proving permission). API Auth vuln = broken/missing/bypassable identity verification on API endpoints.
- Root: API treats request as trusted without validating credentials/tokens correctly, or validates them weakly.
- Distinct from BOLA/BFLA (those assume auth succeeded; this is about auth itself failing).

2. OWASP Category & CWE

- **OWASP API Top 10 2023:** API2:2023 – Broken Authentication
- **OWASP Web Top 10 2021:** A07:2021 – Identification and Authentication Failures
- **CWE:**
 - CWE-287 – Improper Authentication
 - CWE-798 – Use of Hard-coded Credentials
 - CWE-306 – Missing Authentication for Critical Function
 - CWE-521 – Weak Password Requirements
 - CWE-307 – Improper Restriction of Excessive Auth Attempts
 - CWE-613 – Insufficient Session Expiration
 - CWE-347 – Improper Verification of Cryptographic Signature (JWT)
 - CWE-522 – Insufficiently Protected Credentials

3. Common Variants

- Missing auth on endpoint (assume internal/undocumented = safe)
- Weak/no rate limiting on login/OTP → credential stuffing, brute force
- JWT: `alg:none`, algorithm confusion (RS256→HS256), weak secret, no signature verification, no `exp` / `iat` check
- API keys in URL/logs, static/never-rotated keys
- Broken password reset / OTP logic (predictable OTP, no expiry, no attempt limit)
- Basic Auth / API key as sole factor (no MFA) for sensitive ops
- Token not invalidated on logout/password change (no revocation/blacklist)
- Credential leakage via verbose error messages (user enumeration)
- GraphQL introspection/mutations bypassing REST auth middleware
- Auth bypass via alternate path (`/api/v1/admin` vs `/API/V1/Admin`, trailing slash, `.json` suffix)

- mTLS misconfig / client cert not validated

4. Attack Surface / Entry Points

- ☐ Login, registration, password reset, OTP verify endpoints
- ☐ Token issuance endpoint (`/oauth/token` , `/api/auth/login`)
- ☐ Token refresh endpoint
- ☐ API Gateway auth middleware / reverse proxy rules
- ☐ Internal/microservice-to-service endpoints (assumed trusted network)
- ☐ Mobile app hardcoded API keys (APK/IPA decompile)
- ☐ Third-party/partner API keys
- ☐ WebSocket handshake auth
- ☐ GraphQL single endpoint (`/graphql`)
- ☐ Legacy/deprecated API versions (`/v1/` still live, unpatched)
- ☐ Swagger/OpenAPI docs exposing undocumented auth-free routes

5. Testing Workflow (Burp / ZAP / Nuclei unified)

Step 1 — Map all endpoints & auth requirements

- Verify: does every sensitive endpoint enforce auth?
- Manual: crawl app, hit endpoints w/o token
- Burp: Site map + Content Discovery, replay requests in Repeater stripping `Authorization` header
- ZAP: Spider + Forced Browse (OWASP DirBuster lists)
- Nuclei: `exposed-panels` , `exposures` templates; custom template checking 401/403 vs 200 without token
- Payload: send request with no token, empty token, `Bearer null`

Step 2 — Token/JWT structural analysis

- Verify: signature enforced? alg confusion possible? sensitive claims exposed?
- Manual: decode JWT (base64), inspect header/payload
- Burp: JWT Editor extension — tamper `alg` , re-sign
- ZAP: JWT add-on
- Nuclei: `jwt-none-algorithm` , custom regex template for JWT in response
- Payloads: `alg:none` + strip signature; RS256→HS256 using public key as HMAC secret; weak secret brute force (`hashcat` / `jwt_tool`)

Step 3 — Rate limiting / brute force resistance

- Verify: logout, CAPTCHA, throttling on login/OTP
- Manual: scripted repeated login attempts
- Burp: Intruder (cluster bomb/sniper) on login with common creds
- ZAP: Fuzzer

- Nuclei: not ideal (stateful), skip or custom workflow template
- Payload: rockyou.txt subset, OTP 0000–9999 brute force

Step 4 — Session/token lifecycle

- Verify: token invalidated on logout/pwd change; expiry enforced
- Manual: capture token, logout, replay old token
- Burp: Repeater — replay post-logout
- ZAP: manual replay via history
- Nuclei: N/A (needs session state)
- Payload: reuse expired/old token after password reset

Step 5 — Auth bypass via path/method manipulation

- Verify: gateway rule bypass
- Manual: try `/API/admin`, `/admin/`, `/admin%20`, `/admin/.json`, verb tampering (GET→POST/PUT)
- Burp: Repeater — case/verb/path variations
- ZAP: manual + Access Control testing add-on
- Nuclei: `http-missing-security-headers` won't help; custom template for path-bypass patterns
- Payload: `../admin`, double URL-encoding, `X-Original-URL` header

Step 6 — Credential/key exposure

- Verify: keys in JS bundles, mobile binaries, GitHub, logs
- Manual: grep JS/APK for `api_key`, `secret`
- Burp: Logger++ / search history
- ZAP: passive scan alerts
- Nuclei: `exposed-tokens`, `exposed-keys` templates; TruffleHog for repos
- Payload: N/A — recon/OSINT

6. Evidence to Collect

- Full request/response pairs (with/without token) showing 200 OK on protected resource
- Decoded JWT before/after tampering + server acceptance proof
- Screenshot/response showing sensitive data returned unauthenticated
- Rate-limit test logs (N attempts, no lockout, timestamped)
- Token reuse after logout — response showing still-valid session
- HAR file / Burp project file as raw evidence

7. Good PoC Structure

1. Baseline authenticated request (200 OK)
2. Same request with auth removed/tampered (still 200 OK) — screenshot/response body

3. Business impact demonstration — actual sensitive data retrieved or action performed (e.g., another user's PII, admin action executed)
4. Step-by-step reproduction (curl/Burp request) + affected endpoint list
5. Root cause hypothesis (e.g., "gateway validates JWT signature but not `alg` claim")

8. Attack Chains

- Broken Auth → BOLA/IDOR (unauthenticated access to any user's object)
- Broken Auth → BFLA (unauthenticated access to admin functions)
- Credential stuffing → Account Takeover → Stored XSS/CSRF via victim session
- JWT alg confusion → forge admin token → full API takeover
- Leaked API key → SSRF/backend pivot → internal network access
- Weak OTP → ATO → Mass Assignment on profile → privilege escalation

9. False Positives

- 401/403 returned but body still leaks data in error message (not a bypass, but note separately as info disclosure)
- WAF/gateway blocking unauthenticated request in test env but not enforced at app layer (env mismatch)
- Cached response served by CDN mistaken for auth bypass (test with cache-busting headers)
- Token expiry test failing due to clock skew, not actual vuln
- Rate limiting present but only on UI, not API — must test API path directly, not just observe UI behavior

10. Prerequisites for Exploitability

- Endpoint reachable without valid credentials/network restriction (no IP allowlist/mTLS)
- No secondary control (WAF rate-limit, gateway-level auth) compensating
- Token signature/secret discoverable or weak
- Business logic doesn't re-validate identity at data layer (relies solely on auth check)

11. Attacker Achievements

- Full account takeover (any user, including admin)
- Unauthorized data access/exfiltration at scale (no per-user restriction once auth bypassed)
- Unauthorized transactions/actions (financial, admin)
- Lateral movement to internal services (if API key = shared trust boundary)
- Reputational/regulatory exposure via mass PII breach

12. CIA Impact (Tabular, by variant)

Variant	Confidentiality	Integrity	Availability	Example CVSS Vector
Missing auth on endpoint	High	High	Low	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:L
JWT alg:none / weak secret	High	High	Low	AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:N
No rate limit (credential stuffing)	High	Medium	Low	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N
Token not invalidated (logout/reset)	Medium	Medium	None	AV:N/AC:L/PR:L/UI:N/S:U/C:M/I:M/A:N
Hardcoded/leaked API key	High	High	Medium	AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:M

13. Typical CVSS Severity & Vector

- Range: **7.5 – 9.8 (High–Critical)**, esp. if scope-changed (S:C) via privilege escalation
- Representative: `AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H` → 10.0 (unauthenticated full compromise)
- Lower bound (rate-limit only): `AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N` → ~5.3

14. Business Impact (Financial App Context)

- Direct fraud: unauthorized fund transfers/transactions via bypassed auth
- Regulatory: RBI (India) mandates strong customer auth for digital payments; PCI-DSS Req 8 (unique IDs + auth); violations → fines, license scrutiny
- GDPR/DPDP Act: mass unauthenticated PII exposure → breach notification + penalties
- SOX: if financial reporting APIs affected — audit failure
- Reputational: customer trust loss, media coverage disproportionate for banking apps
- Insurance/liability: cyber insurance claims complicated if basic auth controls absent

15. Indicators/Signals

- 200 OK returned on sensitive endpoint without/with malformed `Authorization` header
- JWT signature not verified server-side (accepts tampered payload)
- No `WWW-Authenticate` / consistent 401 across unauthenticated paths
- Login endpoint accepts unlimited attempts (no 429/lockout)
- Same session token valid before/after logout
- API keys visible in JS source, mobile app strings, GitHub repos, Postman collections

- Swagger/OpenAPI publicly accessible listing undocumented routes

16. Root Cause (Code-Level)

- Pseudocode (missing auth):

```
route("/api/admin/users") {  
    // no auth middleware applied  
    return db.getAllUsers()  
}
```

- Pseudocode (JWT flaw):

```
token = parseJWT(header.Authorization)  
if token.payload.role == "admin":    // trusts payload without verifying signature  
    grantAccess()
```

- Common causes: auth middleware not applied globally (per-route opt-in missed), JWT library misconfigured to accept `alg:none`, secret hardcoded/committed to repo, session store not checked on logout, gateway auth $\neq$ app-layer auth (assumed redundant, actually only one enforced).

17. Remediation / Secure Design

- Centralize auth: global middleware/decorator applied by default (deny-by-default, explicit allowlist for public routes)
- JWT: enforce algorithm allowlist server-side (`RS256` only), verify signature always, short `exp`, validate `iss` / `aud`
- Store secrets in vault (not code/repo); rotate keys periodically
- Implement rate limiting + account lockout/backoff on auth endpoints (per-IP + per-account)
- Token revocation: maintain blacklist/short-lived tokens + refresh token rotation
- MFA for sensitive/financial actions
- mTLS or signed requests for service-to-service auth
- Language-agnostic principle: "authenticate first, authorize second, trust nothing from client except after verification"

18. Compensating Controls (if root cause fix delayed)

- WAF rule blocking unauthenticated requests to sensitive paths
- API Gateway-level auth enforcement (even if app-layer weak)
- IP allowlisting for internal/admin APIs
- Enhanced monitoring/alerting on anomalous auth patterns (SIEM rule)

- Temporary rate-limiting at CDN/gateway layer
- Feature-flag disable of vulnerable endpoint until patched

19. Recurrence Signal (SDLC/Posture)

- Indicates no centralized auth framework/library — auth reimplemented per-route inconsistently
- Missing SAST/DAST gate for auth checks in CI/CD
- No security requirements in API design phase (no threat modeling for new endpoints)
- Insufficient code review focus on auth middleware coverage
- Points to systemic gap → mirrors IDOR pattern in your product set (auth = same class of "forgot to check" defect at scale)

20. Upstream Improvements

- SAST: Semgrep taint rule — flag route definitions missing auth decorator/middleware call
- Framework-level: use battle-tested auth middleware (Spring Security, Passport.js, Django REST `IsAuthenticated`) — never hand-roll
- API Gateway: enforce auth policy centrally (Kong/Apigee/AWS API Gateway authorizer) as defense-in-depth
- Design checklist: "every new endpoint PR must specify auth requirement explicitly" (linter/CI check for undecorated routes)
- Shared internal auth library (mirrors your IDOR shared-wrapper strategy) — single source of truth for token validation
- Secrets scanning (gitleaks/TruffleHog) in pre-commit + CI

21. Sample Triage Response

"Confirmed — `/api/v2/account/statement` returns full account data with no `Authorization` header required (tested via Burp Repeater, response 200 with valid JSON). This is unauthenticated access to sensitive financial data (CWE-306, OWASP API2:2023). Severity: Critical (CVSS 9.8, AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N). Escalating to Sev-1; requesting immediate WAF block on endpoint pending fix. PoC and full request/response attached."

22. Sample Developer Remediation Note

"Root cause: `/api/v2/account/statement` route registered outside the global `authMiddleware` group in `routes/account.js` (line 42). Fix: move route under authenticated router group; add `IsAuthenticated` decorator; add unit test asserting 401 for unauthenticated request. Also recommend Semgrep rule to flag any future route registered outside `authMiddleware` scope. Please retest with valid/invalid/missing token cases before closing."

23. Real-World Examples

- **T-Mobile (2023)**: API exposed to exfiltrate ~37M customer records via unauthenticated/under-authenticated endpoint.
- **Peloton API (2021)**: unauthenticated API allowed access to other users' private account data (age, location, etc.) — BOLA compounded by weak auth checks.
- **jsonwebtoken library (CVE-2015-9235)**: `alg:none` acceptance flaw — classic JWT auth bypass root cause still tested for today.
- **Parler (2021)**: sequential, unauthenticated post IDs + lack of proper auth/session controls led to mass scraping post Jan 6 event.